Python vs JavaScript: Code Exploration and Comparison

Script Functionalities and Dataset

The original Python and JavaScript data-cleaning scripts remove duplicates in a case-sensitive manner while preserving capitalisation, sort lists alphabetically, placing uppercase values before lowercase ones, and permanently change the original list order. My script modification supports case-insensitive deduplication and alphabetical sorting without altering the original list order.

I tested my script changes using a Netflix dataset (United States, 2024), focusing on one out of 12 columns – the 'director' column. I first performed a basic cleanup by removing empty and null values, resulting in 6173 rows. Then, after a light alteration of the director names to accommodate various casing variations, the original case-insensitive script produced 4555 rows, and the modified case-sensitive script returned 4526 rows.

Syntax and Structure Comparison

Python syntax is more compact. For instance, the for loop syntax for name in names is concise. The built-in standard library dictionary and the sorting function sorted() are implicit, keeping data processing straightforward and the code simple.

On the contrary, JavaScript syntax, with its constants, semicolons, and curly brackets, increases code length. Choosing the map() method requires more explicit method calls, such as .has() and .set(), to manage the relationship between a key and the values in a map, resulting in more verbose code. The case-insensitive sorting requires an explicit

custom sorting function .sort((a, b) => …) that defines the comparison rules, including overriding default sorting and new ordering, impacting the syntax length.

Ease of Code Understanding and Editing

Python's compact syntax makes code easy to comprehend and edit. It is straightforward to introduce case-sensitive changes and sorting rules because they are concise and easy to read. These exhibit expected behaviour and have clear intent, thereby reducing overcommenting. For instance, sorted() avoids mutating the list, and sorted(key=str.lower) relies on a lowercase key to ensure alphabetical order, ignoring capitalisation.

Conversely, JavaScript requires more deliberate and extensive explanation. For instance, when choosing .sort(), it is crucial to note that it has the side effect of irreversibly altering the data, which then influences the decision to introduce a new array to perform the sorting. This additional complexity requires greater transparency in to the function's behaviour. While explicit logic makes the understanding and editing clearer, it also adds a cognitive load.

Output Differences

Both modified scripts in Python and JavaScript produce the same output. However, the differences can occur when relying on default functions. For instance, Python's sorted() function does not modify the original list, but it returns a new list by default. In contrast, JavaScript's sorting array.sort() mutates the list and sorts it case-sensitively. To keep

behaviour consistent, both implementations ignore capitalisation during comparison and preserve the first name variant encountered.

Conclusion

Overall, Python code is more concise and uses about half as many lines as more verbose JavaScript, making it ideal for straightforward data processing, such as deduplication and sorting. This comparison highlights that the same functionality and similar output can be achieved using different design approaches. Ultimately, the final design choices should be guided by readability, the availability of out-of-the-box libraries, and the need for flexibility and user control over the code design.

Reference

Netflix (United States) (2024) ‘Netflix Title Dataset’. Zenodo. DOI: 10.5281/zenodo.13925131

Appendix

Screenshots of modified Python and JavaScript scripts and their outputs

Python Script

clean_names_modified.py > process_names_modified

```python
# Imports and reads Netlix dataset CSV in pandas
import pandas as pd
df = pd.read_csv('netflix_directors.csv')

cleaned_directors = df['director'].dropna().tolist()
total = len(cleaned_directors)
cleaned = len(set(cleaned_directors))

# Uses only 'director' column and cleans all nulls and empty strings.
# Keeps duplications and casing variations.
print('Total directors count:', total)

# Original Script (case-sensitive)
def process_names(names):

    unique_names = list(set(names))  # Removes duplicates but does not preserve order
    unique_names.sort()              # Sorts alphabetically, changes the original list order
    return unique_names

print("Total unique count directors:", len(process_names(cleaned_directors)))


# Modified Script (case-insensitive)
def process_names_modified(names):

    '''
    Removes duplicated names in a case-insensitive way
    Returns a new list sorted alphabetically, ignoring capitalisation
    '''

    unique = {}                         # Uses dictionary to store unique names
    for name in names:
        lower_name = name.lower()       # Uses lower_name version for comparison
        if lower_name not in unique:    # Converts the name to lowercase for a case-insensitive comparison
            unique[lower_name] = name   # Only stores the unique name if not encountered before. Keeps the first version encountered

    return sorted(unique.values(), key=str.lower) # Sorts unique names in a new list alphabetically, in a case-insensitive way.

# Tests 'Steven Spielberg' casing variations
modified_directors = process_names_modified(cleaned_directors)
target = 'Steven Spielberg'

variations_before = [
    name for name in cleaned_directors
    if name.lower() == target.lower()
]

variations_modified = [
    name for name in modified_directors
    if name.lower() == target.lower()

 ]
```

Python Output

```
Total directors count: 6173
Total unique count directors: 4555
Count cleaned variations: 11
Cleaned list variations:
- steven Spielberg
- STEVEN Spielberg
- Steven Spielberg
- Steven Spielberg
- Steven Spielberg
- STEVEN SPIELBERG
- steven SPIELBERG
- Steven Spielberg
- steven spielberg
- Steven Spielberg
- Steven Spielberg
Count modified variation: 1
Modified list variation:
- steven Spielberg
Total unique count directors: 4526 _
```

JavaScript Script

clean_names_modified.py ● JS cleanNames_modified.js ●

JS cleanNames_modified.js > ...

```javascript
// Imports and read Netlix CSV dataset via Papaparse
import fs from 'fs';
import Papa from 'papaparse';

const csvText = fs.readFileSync('./netflix_directors.csv', 'utf8');

const parsed = Papa.parse(csvText, {
    header: true,
    skipEmptyLines: true
});

// Uses only 'director' column and cleans all nulls and empty strings.
// Keeps duplications and ignores letter casing.
const netflix_directors = parsed.data
const raw_directors = netflix_directors.map(row => row.director)
const cleaned_directors = raw_directors.filter(director => director !== null && director !== '');

console.log('Total directors count:', cleaned_directors.length)

//Original Script (case-sensitive)

function processNames(names) {
    let uniqueNames = [...new Set(names)]; // removes duplicates in case-sensitive way
    uniqueNames.sort(); // sorts alphabetically, changes the original list order
    return uniqueNames;
}

console.log('Total unique directors count:', processNames(cleaned_directors).length);

```

```javascript

// Modified Script (case-insensitive)

    // Removes duplicated names in a case-insensitive way
    // Returns a new array sorted alphabetically, ignoring capitalisation
    // Preserves the input in the original array

function processNames_modified(names)
{
    const unique = new Map();                   // Creates a new Map to store pairs, lowercase and original name
    for (const name of names) {
        const lower_name = name.toLowerCase();  // Converts the original name to lowercase for a case-insensitive comparison
        if (!unique.has(lower_name)){
            unique.set(lower_name, name);       // Only stores the unique name if not encountered before. Keeps the first version encountered
        }
    }
    return Array.from(unique.values())          // Converts Map values, to a new array before sorting to avoid array mutation
        .sort((a, b) => a.toLowerCase().localeCompare(b.toLowerCase())); // Sorts names in the new array alphabetically in a case-insensitive way
    }

```

```javascript
// Tests 'Steven Spielberg' casing variations
const modified_directors = processNames_modified(cleaned_directors);
const target = 'Steven Spielberg';

const variations_before = cleaned_directors.filter(
    name => name.toLowerCase() === target.toLowerCase()
);

const variations_modified = modified_directors.filter(
    name => name.toLowerCase() === target.toLowerCase()
);

console.log ('Count cleaned variations:', variations_before.length);
console.log ('Cleaned list variations:', variations_before);
console.log ('Count modified variation:', variations_modified.length);
console.log ('Modified list variation:', variations_modified);

console.log('Total unique directors count:', modified_directors.length)
```

JavaScript Output

```
Total directors count: 6173
Total unique directors count: 4555
Count cleaned variations: 11
Cleaned list variations: [
  'steven Spielberg',
  'STEVEN Spielberg',
  'Steven Spielberg',
  'Steven Spielberg',
  'Steven Spielberg',
  'STEVEN SPIELBERG',
  'steven SPIELBERG',
  'Steven Spielberg',
  'steven spielberg',
  'Steven Spielberg',
  'Steven Spielberg'
]
Count modified variation: 1
Modified list variation: [ 'steven Spielberg' ]
Total unique directors count: 4526
```